

## **Epik 1 – Zarządzanie użytkownikami**

### **US1 – Dodawanie kursantów**

#### **Cel:**

Umożliwienie administratorowi oraz korepetytorowi dodawania nowych kursantów do systemu.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator albo korepetytor.
- Użytkownik ma dostęp do widoku kursantów.
- Formularz dodawania kursanta jest dostępny z poziomu panelu.

###### **2. Proces**

- Użytkownik uzupełnia imię, nazwisko, e-mail i numer telefonu.
- System sprawdza kompletność danych.
- System sprawdza poprawność formatu e-maila.
- System sprawdza, czy kursant o takim e-mailu już istnieje.

###### **3. Warunki końcowe**

- Kursant zostaje zapisany w systemie.
- Kursant pojawia się na liście kursantów.

##### **b) Niefunkcjonalne**

- Formularz powinien być prosty i czytelny.
- Komunikaty błędów powinny jednoznacznie wskazywać problem.
- Dodanie kursanta powinno być widoczne natychmiast po zapisaniu.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Poprawne dodanie kursanta**

**Given:** Użytkownik jest w formularzu dodawania kursanta.

**When:** Wpisuje poprawne dane i klika „Zapisz”.

**Then:** System zapisuje kursanta i pokazuje go na liście kursantów.

##### **Scenariusz 2: Brak wymaganych danych**

**Given:** Użytkownik otworzył formularz dodawania kursanta.

**When:** Pozostawia puste wymagane pole.

**Then:** System wyświetla błąd i nie zapisuje kursanta.

##### **Scenariusz 3: Duplikat kursanta**

**Given:** W systemie istnieje kursant z danym e-mailem.

**When:** Użytkownik próbuje dodać kursanta z tym samym e-mailem.

**Then:** System informuje o istniejącym kursancie i blokuje zapis.

### **US2 – Edycja danych kursantów**

#### **Cel:**

Umożliwienie administratorowi lub korepetytorowi poprawiania danych kursantów, np. numeru telefonu lub e-maila.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator albo korepetytor.
- W systemie istnieje co najmniej jeden kursant.
- Użytkownik ma dostęp do listy kursantów.

## **2. Proces**

- Użytkownik wybiera kursanta z listy.
- System otwiera formularz edycji z aktualnymi danymi.
- Użytkownik zmienia dane i zapisuje formularz.
- System sprawdza poprawność danych.

## **3. Warunki końcowe**

- Dane kursanta zostają zaktualizowane.
- Zmiany są widoczne na liście kursantów.

## **b) Niefunkcjonalne**

- Formularz edycji powinien być spójny z formularzem dodawania kursanta.
- System powinien jasno wskazywać błędne pola.

## **Scenariusze testowe:**

### **Scenariusz 1: Poprawna edycja kursanta**

**Given:** Użytkownik otworzył formularz edycji kursanta.

**When:** Zmienia numer telefonu i klika „Zapisz”.

**Then:** System zapisuje zmianę i pokazuje aktualne dane na liście.

### **Scenariusz 2: Niepoprawny adres e-mail**

**Given:** Użytkownik edytuje dane kursanta.

**When:** Wpisuje e-mail w niepoprawnym formacie.

**Then:** System wyświetla błąd i nie zapisuje zmian.

## **US3 – Dodawanie korepetytorów**

### **Cel:**

Umożliwienie administratorowi tworzenia kont korepetytorów w systemie.

### **Kryteria akceptacji:**

## **a) Funkcjonalne**

### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator.
- Administrator ma dostęp do widoku korepetytorów.
- Formularz dodawania korepetytora jest dostępny z panelu administratora.

## **2. Proces**

- Administrator wpisuje imię, nazwisko, e-mail i telefon korepetytora.
- System sprawdza kompletność i poprawność danych.
- System sprawdza, czy korepetytor nie istnieje już w systemie.

## **3. Warunki końcowe**

- Korepetytor zostaje dodany do systemu.
- Korepetytor pojawia się na liście korepetytorów.
- Korepetytor może zostać przypisany do grupy.

## **b) Niefunkcjonalne**

- Funkcja dodawania korepetytorów powinna być dostępna tylko dla administratora.
- Formularz powinien być prosty i spójny z pozostałymi formularzami.

## **Scenariusze testowe GWT**

### **Scenariusz 1: Dodanie korepetytora przez administratora**

**Given:** Administrator znajduje się w formularzu dodawania korepetytora.

**When:** Wpisuje poprawne dane i klika „Zapisz”.

**Then:** System dodaje korepetytora i pokazuje go na liście.

#### **Scenariusz 2: Brak wymaganych danych**

**Given:** Administrator otworzył formularz dodawania korepetytora.

**When:** Pozostawia puste pole e-mail.

**Then:** System pokazuje błąd i blokuje zapis.

#### **Scenariusz 3: Duplikat korepetytora**

**Given:** W systemie istnieje korepetytor z danym e-mailem.

**When:** Użytkownik próbuje dodać korepetytora z tym samym e-mailem.

**Then:** System informuje o istniejącym korepetytorze i blokuje zapis.

### **US4 – Edycja danych korepetytorów**

#### **Cel:**

Umożliwienie administratorowi poprawiania danych korepetytorów, np. numeru telefonu lub e-maila.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator.
- W systemie istnieje co najmniej jeden korepetytor.

###### **2. Proces**

- Użytkownik wybiera korepetytora z listy.
- System otwiera formularz edycji z aktualnymi danymi.
- Użytkownik zmienia dane i zapisuje formularz.
- System sprawdza poprawność danych.

###### **3. Warunki końcowe**

- Dane korepetytora zostają zaktualizowane.
- Zmiany są widoczne na liście korepetytorów.

##### **b) Niefunkcjonalne**

- Formularz edycji powinien być spójny z formularzem dodawania korepetytora.
- System powinien jasno wskazywać błędne pola.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Poprawna edycja korepetytora**

**Given:** Użytkownik otworzył formularz edycji korepetytora.

**When:** Zmienia numer telefonu i klika „Zapisz”.

**Then:** System zapisuje zmianę i pokazuje aktualne dane na liście.

##### **Scenariusz 2: Niepoprawny adres e-mail**

**Given:** Użytkownik edytuje dane korepetytora.

**When:** Wpisuje e-mail w niepoprawnym formacie.

**Then:** System wyświetla błąd i nie zapisuje zmian.

##### **Scenariusz 3: Anulowanie edycji**

**Given:** Użytkownik znajduje się w formularzu edycji.

**When:** Klika „Anuluj”.

**Then:** System wraca do listy korepetytorów bez zmiany danych.

## **Epik 2 – Zarządzanie grupami**

### **US5 – Tworzenie grup zajęciowych**

#### **Cel:**

Utworzenie grupy z nazwą, poziomem, tematem, prowadzącym i listą kursantów.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator albo korepetytor.
- W systemie istnieją kursanci.
- W przypadku administratora istnieje lista korepetytorów.

###### **2. Proces**

- Użytkownik podaje nazwę, poziom i temat grupy.
- Administrator wybiera prowadzącego z listy korepetytorów / korepetytor tworzący grupę zostaje automatycznie prowadzącym.
- Użytkownik może przypisać kursantów do grupy.

###### **3. Warunki końcowe**

- Grupa zostaje utworzona.
- Grupa pojawia się na liście grup.
- Administrator widzi wszystkie grupy, a korepetytor tylko swoje.

##### **b) Niefunkcjonalne**

- Formularz tworzenia grupy powinien być przejrzysty.
- Lista kursantów powinna być łatwa do przeglądania.
- Utworzenie grupy powinno być widoczne od razu po zapisie.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Administrator tworzy grupę**

**Given:** Administrator jest w formularzu tworzenia grupy.

**When:** Podaje dane grupy, wybiera korepetytora i zapisuje.

**Then:** System tworzy grupę z wybranym prowadzącym.

##### **Scenariusz 2: Korepetytor tworzy własną grupę**

**Given:** Korepetytor jest zalogowany.

**When:** Tworzy grupę i zapisuje formularz.

**Then:** System tworzy grupę i przypisuje go jako prowadzącego.

##### **Scenariusz 3: Brak nazwy grupy**

**Given:** Użytkownik tworzy grupę.

**When:** Nie podaje nazwy i klika „Zapisz”.

**Then:** System wyświetla błąd i nie tworzy grupy.

### **US6 – Przypisywanie kursantów do grup**

#### **Cel:**

Zarządzanie składem grup zajęciowych.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- W systemie istnieje grupa.
- W systemie istnieją kursanci.
- Użytkownik ma uprawnienia do edycji danej grupy.

## **2. Proces**

- Użytkownik otwiera zarządzanie lub formularz grupy.
- Wybiera kursantów z listy.
- Zapisuje skład grupy.

## **3. Warunki końcowe**

- Kursanci zostają przypisani do grupy.
- Lista uczestników grupy zostaje zaktualizowana.

### **b) Niefunkcjonalne**

- Lista uczestników powinna być czytelna.
- Zmiany w składzie grupy powinny być widoczne natychmiast.
- System powinien chronić dane grup innych korepetytorów.

### **Scenariusze testowe:**

#### **Scenariusz 1: Przypisanie kursanta do grupy**

**Given:** Użytkownik ma dostęp do edycji grupy.

**When:** Wybiera kursanta i zapisuje zmiany.

**Then:** System dodaje bądź usuwa kursanta z listy uczestników grupy zależnie od wyboru.

## **US7 – Przypisywanie korepetytorów do grup**

### **Cel:**

Zapewnienie, że każda grupa ma przypisanego korepetytora odpowiedzialnego za zajęcia.

### **Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator.
- W systemie istnieje co najmniej jeden korepetytor.
- Administrator tworzy lub edytuje grupę.

##### **2. Proces**

- Administrator wybiera korepetytora z listy.
- System przypisuje go do grupy.
- System zapisuje dane grupy.

##### **3. Warunki końcowe**

- Grupa ma przypisanego prowadzącego.
- Prowadzący jest widoczny w szczegółach grupy.
- Grupa pojawia się w panelu przypisanego korepetytora.

#### **b) Niefunkcjonalne**

- Lista korepetytorów powinna być czytelna.

### **Scenariusze testowe:**

#### **Scenariusz 1: Przypisanie korepetytora przy tworzeniu grupy**

**Given:** Administrator tworzy nową grupę.

**When:** Wybiera korepetytora i zapisuje grupę.

**Then:** System przypisuje wybranego korepetytora jako prowadzącego.

#### **Scenariusz 2: Próba utworzenia grupy bez prowadzącego**

**Given:** Administrator tworzy grupę.

**When:** Nie wybiera korepetytora i klika „Zapisz”.

**Then:** System wyświetla błąd i blokuje utworzenie grupy.

### **Scenariusz 3: Grupa widoczna dla prowadzącego**

**Given:** Administrator przypisał korepetytora do grupy.

**When:** Korepetytor otwiera widok swoich grup.

**Then:** System pokazuje przypisaną grupę.

## **Epik 3 – Harmonogram**

### **US8 – Tworzenie harmonogramu**

**Cel:**

Dodawanie zajęć do harmonogramu.

**Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- Użytkownik jest zalogowany jako administrator albo korepetytor.
- Istnieje co najmniej jedna grupa.
- Korepetytor ma dostęp tylko do grup prowadzonych przez siebie.

##### **2. Proces**

- Użytkownik wybiera grupę, datę oraz godziny zajęć.
- System sprawdza poprawność czasu.
- System sprawdza konflikty terminów.
- System może zasugerować termin ograniczający „okienka”.

##### **3. Warunki końcowe**

- Zajęcia zostają zapisane w harmonogramie.
- Zajęcia są widoczne dla prowadzącego i kursantów grupy.
- System generuje powiadomienie o nowych zajęciach.

#### **b) Niefunkcjonalne**

- Dodawanie zajęć powinno wymagać niewielu kroków.

**Scenariusze testowe:**

#### **Scenariusz 1: Administrator dodaje zajęcia**

**Given:** Administrator znajduje się w formularzu dodawania zajęć.

**When:** Wybiera grupę, datę i poprawny przedział godzin.

**Then:** System dodaje zajęcia do harmonogramu.

#### **Scenariusz 2: Korepetytor dodaje zajęcia dla swojej grupy**

**Given:** Korepetytor prowadzi wybraną grupę.

**When:** Dodaje zajęcia dla tej grupy.

**Then:** System zapisuje zajęcia i pokazuje je w jego harmonogramie.

#### **Scenariusz 3: Niepoprawny czas zajęć**

**Given:** Użytkownik dodaje zajęcia.

**When:** Podaje godzinę zakończenia wcześniejszą niż rozpoczęcia.

**Then:** System wyświetla błąd i nie zapisuje zajęć.

#### **Scenariusz 4: Sugestia ograniczająca okienka**

**Given:** Korepetytor ma już zajęcia w danym dniu.

**When:** Użytkownik dodaje kolejne zajęcia.

**Then:** System sugeruje termin przed lub po istniejących zajęciach.

## **US9 – Modyfikacja harmonogramu**

### **Cel:**

Umożliwienie usunięcia istniejących zajęć.

### **Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- W systemie istnieją zaplanowane zajęcia.
- Użytkownik ma uprawnienia do ich edycji.

##### **2. Proces**

- Użytkownik otwiera szczegóły zajęć.
- Użytkownik usuwa zajęcia.
- System sprawdza poprawność zmian.

##### **3. Warunki końcowe**

- Harmonogram zostaje zaktualizowany.
- Zainteresowani użytkownicy otrzymują powiadomienie.

#### **b) Niefunkcjonalne**

- Zmiana harmonogramu powinna być widoczna natychmiast.
- System powinien wymagać potwierdzenia usunięcia zajęć.

### **Scenariusze testowe**

#### **Scenariusz 1: Usunięcie zajęć**

**Given:** Użytkownik ma uprawnienia do edycji zajęć.

**When:** Klika „Usuń” i potwierdza operację.

**Then:** System usuwa zajęcia z harmonogramu.

#### **Scenariusz 2: Powiadomienie po zmianie**

**Given:** Zajęcia zostały usunięte.

**When:** System zapisuje zmianę.

**Then:** Kursanci otrzymują powiadomienie.

## **US10 – Wykrywanie konfliktów**

### **Cel:**

Zapobieganie sytuacjom, w których korepetytor, grupa lub kursant mają kilka zajęć w tym samym czasie.

### **Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- W systemie istnieją już zaplanowane zajęcia.
- Użytkownik ma uprawnienia i dodaje zajęcia.

##### **2. Proces**

- System porównuje nowy termin z istniejącymi zajęciami.
- System sprawdza konflikt korepetytora.
- System sprawdza konflikt grupy.
- System sprawdza konflikt kursantów.

##### **3. Warunki końcowe**

- W razie konfliktu system pokazuje komunikat.

- Komunikat wskazuje, czego dotyczy konflikt.
- System blokuje zapis konfliktowego terminu.

#### **b) Niefunkcjonalne**

- Komunikat konfliktu powinien być zrozumiały.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Konflikt korepetytora**

**Given:** Korepetytor ma już zajęcia w danym czasie.

**When:** Użytkownik próbuje dodać mu inne zajęcia w tym samym czasie.

**Then:** System pokazuje konflikt korepetytora i blokuje zapis.

##### **Scenariusz 2: Konflikt grupy**

**Given:** Grupa ma zaplanowane zajęcia w danym czasie.

**When:** Użytkownik próbuje dodać kolejne zajęcia tej grupy w tym samym czasie.

**Then:** System pokazuje konflikt grupy i blokuje zapis.

##### **Scenariusz 3: Konflikt kursanta**

**Given:** Kursant należy do dwóch grup.

**When:** Użytkownik planuje zajęcia w taki sposób, że obie grupy mają w tym samym czasie.

**Then:** System informuje o konflikcie kursanta.

##### **Scenariusz 4: Brak konfliktu**

**Given:** Wybrany termin nie koliduje z innymi zajęciami.

**When:** Użytkownik zapisuje zajęcia.

**Then:** System dodaje je do harmonogramu.

## **Epik 4 – Powiadomienia**

### **US11 – Powiadamianie użytkowników**

#### **Cel:**

Automatyczne informowanie użytkowników o zmianach dotyczących ich grup, zajęć lub zapisów.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- W systemie istnieją użytkownicy przypisani do grup.
- Użytkownicy mają dostęp do widoku powiadomień.
- Występuje zdarzenie wymagające powiadomienia.

###### **2. Proces**

- System wykrywa dodanie lub usunięcie zajęć.
- System wykrywa zapisanie lub wypisanie kursanta.
- System tworzy powiadomienie.
- System przypisuje powiadomienie właściwym użytkownikom.

###### **3. Warunki końcowe**

- Powiadomienie pojawia się w widoku powiadomień.
- Otrzymują je tylko zainteresowani użytkownicy.

##### **b) Niefunkcjonalne**

- Powiadomienia powinny pojawiać się jak najszybciej po wywołującym je zdarzeniu.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Powiadomienie po dodaniu lub usunięciu zajęć**

**Given:** Administrator lub korepetytor dodaje lub usuwa zajęcia dla grupy.

**When:** System zapisuje zajęcia.

**Then:** Kursanci grupy i prowadzący otrzymują powiadomienie.

### **Scenariusz 2: Powiadomienie po zapisaniu lub wypisaniu się kursanta**

**Given:** Kursant zapisuje lub wypisuje się z zajęć.

**When:** System zapisuje wybór.

**Then:** Kursant otrzymuje potwierdzenie.

## **US12 – Otrzymywanie powiadomień**

### **Cel:**

Zapewnienie użytkownikowi dostępu do informacji o zmianach związanych z jego zajęciami.

### **Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- Użytkownik jest zalogowany.
- Użytkownik ma dostęp do widoku swoich powiadomień.

##### **2. Proces**

- Użytkownik otwiera widok powiadomień.
- System pokazuje powiadomienia użytkownika w kolejności chronologicznej.
- Użytkownik odczytuje szczegóły powiadomienia.

##### **3. Warunki końcowe**

- Po odczytaniu powiadomienie zostaje oznaczone jako przeczytane.
- Po kliknięciu "Usuń" powiadomienie znika z listy powiadomień.

#### **b) Niefunkcjonalne**

- Nieprzeczytane powiadomienia powinny być wyróżnione.
- Treść powiadomienia powinna być krótka i zrozumiała.
- Lista powiadomień powinna być czytelna.
- Użytkownik nie powinien widzieć cudzych powiadomień.

### **Scenariusze testowe:**

#### **Scenariusz 1: Podgląd powiadomień**

**Given:** Użytkownik ma przypisane powiadomienia.

**When:** Otwiera widok „Powiadomienia”.

**Then:** System pokazuje listę jego powiadomień.

#### **Scenariusz 2: Oznaczenie nieprzeczytanego powiadomienia**

**Given:** Użytkownik ma nowe powiadomienie.

**When:** Otwiera panel systemu.

**Then:** System wyróżnia nieprzeczytane powiadomienia.

#### **Scenariusz 3: Usunięcie powiadomienia**

**Given:** Użytkownik jest w widoku powiadomień.

**When:** Użytkownik klika usunięcie powiadomienia.

**Then:** System usuwa powiadomienie z widoku użytkownika.

## **Epik 5 – Zapisy na zajęcia**

### **US13 – Zapisy na zajęcia**

**Cel:**

Umożliwienie kursantowi samodzielnego wyboru i zapisu na dostępne zajęcia.

**Kryteria akceptacji:****a) Funkcjonalne****1. Warunki wstępne**

- Użytkownik jest zalogowany jako kursant.
- W systemie istnieją dostępne zajęcia.
- Kursant ma dostęp do widoku zapisów.

**2. Proces**

- Kursant przegląda dostępne zajęcia.
- Kursant wybiera zajęcia i potwierdza zapis.
- System sprawdza, czy są wolne miejsca lub nie występuje konflikt.
- System zapisuje kursanta na zajęcia.

**3. Warunki końcowe**

- Zajęcia pojawiają się w harmonogramie kursanta.
- System generuje powiadomienie o zapisie.
- Kursant otrzymuje potwierdzenie o zapisie.

**b) Niefunkcjonalne**

- Widok zapisów powinien być prosty i zrozumiały.
- Proces zapisu powinien wymagać małej liczby kroków.

**Scenariusze testowe:****Scenariusz 1: Kursant zapisuje się na zajęcia**

**Given:** Kursant widzi dostępne zajęcia.

**When:** Klika „Zapisz się” i potwierdza wybór.

**Then:** System zapisuje kursanta na zajęcia i zmniejsza liczbę dostępnych miejsc.

**Scenariusz 2: Zajęcia pojawiają się w harmonogramie**

**Given:** Kursant zapisał się na zajęcia.

**When:** Otwiera swój harmonogram.

**Then:** System pokazuje nowe zajęcia w harmonogramie kursanta.

**Scenariusz 3: Próba zapisu na zajęcia bez wolnych miejsc**

**Given:** Kursant widzi dostępne zajęcia.

**When:** Próbuje zapisać się na zajęte zajęcia.

**Then:** System blokuje operację i pokazuje komunikat.

**Scenariusz 4: Konflikt terminu**

**Given:** Kursant próbuje zapisać się na dane zajęcia.

**When:** Ma inne zajęcia w podanym terminie.

**Then:** System blokuje operację i pokazuje komunikat o konflikcie.

**US14 – Rezygnacja z zajęć****Cel:**

Umożliwienie kursantowi samodzielnej rezygnacji z zajęć.

**Kryteria akceptacji:****a) Funkcjonalne****1. Warunki wstępne**

- Kursant jest zalogowany.

- Kursant jest zapisany na dane zajęcia.
- Kursant jest w widoku harmonogramu.

## **2. Proces**

- Kursant wybiera dane zajęcia.
- Kursant klika „Wypisz się”.
- System usuwa zapis kursanta.

## **3. Warunki końcowe**

- Zajęcia znikają z harmonogramu kursanta.
- Zajęcia pozostają w harmonogramie szkoły.
- System generuje powiadomienie o rezygnacji.

### **b) Niefunkcjonalne**

- Przycisk wypisania powinien być widoczny tylko przy przyszłych zajęciach.
- System powinien wymagać potwierdzenia rezygnacji.

### **Scenariusze testowe:**

#### **Scenariusz 1: Wypisanie się z zajęć**

**Given:** Kursant jest zapisany na dane zajęcia.

**When:** Klika „Wypisz się” i potwierdza decyzję.

**Then:** System usuwa zapis kursanta z zajęć.

#### **Scenariusz 2: Brak możliwości wypisania się z zakończonych zajęć**

**Given:** Zajęcia już się odbyły.

**When:** Kursant otwiera historię zajęć.

**Then:** System nie pokazuje opcji „Wypisz się”.

## **Epik 6 – Dostęp do danych**

### **US15 – Podgląd harmonogramu przez korepetytora**

#### **Cel:**

Zapewnienie korepetytorowi dostępu do aktualnego harmonogramu jego grup.

#### **Kryteria akceptacji:**

### **a) Funkcjonalne**

#### **1. Warunki wstępne**

- Korepetytor jest zalogowany.
- Korepetytor prowadzi co najmniej jedną grupę.
- W systemie istnieją zajęcia jego grup.

#### **2. Proces**

- Korepetytor otwiera widok harmonogramu.
- System pobiera zajęcia przypisane do jego grup.
- System prezentuje je w widoku graficznym.

#### **3. Warunki końcowe**

- Korepetytor widzi tylko swoje zajęcia.
- Korepetytor może sprawdzić szczegóły zajęć.

### **b) Niefunkcjonalne**

- Harmonogram powinien mieć czytelny widok graficzny niezależnie od urządzenia.
- Zajęcia powinny być przedstawione jako bloki czasowe zawierające godzinę i nazwę grupy.
- Dane powinny być aktualne po zmianach.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Korepetytor widzi swoje zajęcia**

**Given:** Korepetytor prowadzi kilka grup.

**When:** Otwiera harmonogram.

**Then:** System pokazuje zajęcia jego grup.

##### **Scenariusz 2: Kliknięcie bloku zajęć**

**Given:** Korepetytor widzi blok zajęć w harmonogramie.

**When:** Klika blok.

**Then:** System pokazuje szczegóły zajęć.

#### **US16 – Lista uczestników**

##### **Cel:**

Umożliwienie korepetytorowi sprawdzenia uczestników swoich grup.

##### **Kryteria akceptacji:**

###### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Korepetytor jest zalogowany.
- Korepetytor prowadzi co najmniej jedną grupę.

###### **2. Proces**

- Korepetytor otwiera widok swoich grup.
- System pobiera kursantów z jego grup.
- System wyświetla podstawowe dane kursantów.

###### **3. Warunki końcowe**

- Korepetytor widzi kursantów swoich grup.

###### **b) Niefunkcjonalne**

- System powinien chronić dane kursantów spoza grup korepetytora.
- Widok powinien działać sprawnie przy większej liczbie kursantów.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Podgląd uczestników własnej grupy**

**Given:** Korepetytor prowadzi grupę z kursantami.

**When:** Otwiera listę grup.

**Then:** System pokazuje kursantów z grup.

#### **US17 – Podgląd harmonogramu przez kursanta**

##### **Cel:**

Zapewnienie kursantowi dostępu do aktualnego planu zajęć.

##### **Kryteria akceptacji:**

###### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Kursant jest zalogowany.
- Kursant ma dostęp do widoku harmonogramu.

###### **2. Proces**

- Kursant otwiera harmonogram.
- System pobiera jego zajęcia.

- System prezentuje zajęcia w widoku graficznym.

### **3. Warunki końcowe**

- Kursant widzi swoje zajęcia uporządkowane chronologicznie.
- Kursant może podejrzeć szczegóły zajęć.

#### **b) Niefunkcjonalne**

- Harmonogram powinien być czytelny.
- Widok powinien umożliwiać szybkie sprawdzenie najbliższych zajęć.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Kursant widzi swoje zajęcia**

**Given:** Kursant jest zapisany na zajęcia.

**When:** Otwiera harmonogram.

**Then:** System pokazuje jego zajęcia.

## **US18 – Historia uczestnictwa**

#### **Cel:**

Pokazanie kursantowi zakończonych oraz przyszłych zajęć wraz ze statusem.

#### **Kryteria akceptacji:**

##### **a) Funkcjonalne**

###### **1. Warunki wstępne**

- Kursant jest zalogowany.
- Kursant uczestniczył lub jest zapisany na zajęcia.
- Kursant ma dostęp do widoku historii.

###### **2. Proces**

- Kursant otwiera widok historii.
- System pobiera zajęcia powiązane z kursantem.
- System rozróżnia zajęcia zakończone i przyszłe.

###### **3. Warunki końcowe**

- Kursant widzi własną historię zajęć w kolejności chronologicznej.
- Zajęcia zakończone nie mają opcji wypisania.
- Przy przyszłych zajęciach dostępna jest możliwość rezygnacji.

##### **b) Niefunkcjonalne**

- Widok historii powinien działać sprawnie przy większej liczbie wpisów.

#### **Scenariusze testowe:**

##### **Scenariusz 1: Podgląd historii zajęć**

**Given:** Kursant ma zakończone zajęcia.

**When:** Otwiera widok historii.

**Then:** System pokazuje listę zakończonych zajęć.

##### **Scenariusz 2: Rozróżnienie przyszłych i zakończonych zajęć**

**Given:** Kursant ma zajęcia przyszłe i zakończone.

**When:** Otwiera historię.

**Then:** System pokazuje odpowiednie statusy zajęć.

##### **Scenariusz 3: Rezygnacja z przyszłych zajęć z poziomu historii**

**Given:** Kursant ma przyszłe zajęcia w historii.

**When:** Klika „Wypisz się” i potwierdza decyzję.

**Then:** System usuwa zapis kursanta z tych zajęć.

## **US19 – Logowanie do systemu**

### **Cel:**

Umożliwienie użytkownikowi bezpiecznego dostępu do systemu oraz rozpoznanie jego roli.

### **Kryteria akceptacji:**

#### **a) Funkcjonalne**

##### **1. Warunki wstępne**

- Użytkownik posiada konto w systemie.
- Użytkownik znajduje się na stronie logowania.
- System jest dostępny online.

##### **2. Proces**

- Użytkownik wprowadza e-mail i hasło.
- Użytkownik zatwierdza formularz logowania.
- System sprawdza poprawność danych logowania.
- System rozpoznaje rolę użytkownika.

##### **3. Warunki końcowe**

- Przy poprawnych danych użytkownik zostaje zalogowany.
- Użytkownik zostaje przekierowany do odpowiedniego panelu:
  - administrator → panel administracyjny
  - korepetytor → panel korepetytora
  - kursant → panel kursanta
- Przy błędnych danych system wyświetla komunikat o błędzie.
- System nie ujawnia szczegółów (np. czy login istnieje).

#### **b) Niefunkcjonalne**

- Hasło nie może być widoczne podczas wpisywania (maskowanie).
- System powinien być odporny na próby zgadywania haseł (np. limit prób)..
- Logowanie powinno działać szybko (np. <2 sekundy).
- Sesja użytkownika powinna być utrzymywana po zalogowaniu.

### **Scenariusze testowe:**

#### **Scenariusz 1: Poprawne logowanie**

Given: Użytkownik znajduje się na stronie logowania i ma poprawne dane.

When: Wprowadza poprawny e-mail i hasło oraz klika „Zaloguj”.

Then: System loguje użytkownika i przekierowuje go do odpowiedniego panelu.

#### **Scenariusz 2: Błędne hasło**

Given: Użytkownik znajduje się na stronie logowania.

When: Wprowadza poprawny e-mail i błędne hasło.

Then: System wyświetla komunikat o niepoprawnych danych logowania.

#### **Scenariusz 3: Nieistniejący użytkownik**

Given: Użytkownik znajduje się na stronie logowania.

When: Wprowadza e-mail, który nie istnieje w systemie.

Then: System wyświetla ogólny komunikat o błędnych danych logowania.

#### **Scenariusz 4: Puste pola**

Given: Użytkownik otworzył formularz logowania.

When: Kliknie „Zaloguj” bez uzupełnienia danych.

Then: System wyświetla komunikat o wymaganych polach.

#### **Scenariusz 5: Przekierowanie zależne od roli**

Given: Użytkownik ma przypisaną rolę (np. korepetytor).

When: Loguje się poprawnie.

Then: System przekierowuje go do panelu odpowiedniego dla tej roli.

**Ogólne wymagania niefunkcjonalne:**

- system powinien być przejrzysty i intuicyjny w obsłudze dla użytkowników
- najważniejsze funkcje (harmonogram, zapisy, powiadomienia) powinny być dostępne w maksymalnie 2–3 kliknięciach
- system powinien działać poprawnie zarówno na komputerze, jak i urządzeniach mobilnych
- czas ładowania głównych widoków (np. harmonogramu) nie powinien przekraczać 2 sekund
- aplikacja powinna być skalowalna horyzontalnie i wertykalnie
- operacje (np. zapis na zajęcia) powinny być realizowane w czasie krótszym niż 1 sekunda
- system powinien być dostępny przez co najmniej 99% czasu
- dane użytkowników nie mogą być utracone w wyniku błędów systemu
- dostęp do danych powinien być ograniczony zgodnie z rolą użytkownika (administrator, korepetytor, kursant) oraz być chroniony przed nieautoryzowanym dostępem
- system powinien wymagać logowania do konta
- system powinien działać w popularnych przeglądarkach (Chrome, Firefox, Edge)